Defensa del proyecto: explicación del código

Zombis vs. Plantas · Proyecto 2 · IPC1 · 2S2026

Guía para comprender qué hace cada parte, cómo se conectan las clases y qué sucede con los datos al ejecutar el programa. Incluye fragmentos reales y ejemplos para practicar la explicación.

Corresponde a la versión corregida del repositorio. Los números de línea se refieren a ese código; si lo editaste, localiza los métodos con Ctrl + F. Los fragmentos largos se ajustan visualmente al ancho de la página.

Qué vas a explicar	Página	Apartado evaluado
Arranque y recorrido de llamadas	2	POO y navegación
Validación y persistencia de cuentas	3	Cuentas: 8 puntos en total
Lectura propia del JSON	4	Configuración: 8 puntos en total
Matriz, casillas y estado de la partida	5	Construcción del jardín
Del clic a una compra	6	Invocación y recursos
Reloj y ciclo del motor	7	Motor: 15 puntos en total
Clases y lógica de plantas / zombis	8–9	14 puntos por catálogo
Movimiento, impactos y casilla lógica	10	Combate y reglas espaciales
Generación y recogida de cerebros	11	Economía
Defensa de la computadora	12	Dificultad e IA: 8 puntos
Bloques, descuentos, podadoras y fin	13	Hordas y final: 7 puntos
Trabajadores y EDT de Swing	14	Implementación técnica
Pausa y serialización	15	Guardado: 6 puntos
Estadísticas y ordenamiento	16	Estadísticas: 6 puntos
Ventanas, dibujo e imágenes	17	Interfaz y arquitectura
Consola, preguntas y demostración	18	Repaso y documentación: 10 puntos

Cómo estudiar: para cada fragmento identifica entrada, condición, cambio de estado y resultado visible. Después explícalo sin leerlo literalmente y compruébalo ejecutando el juego.

1. Cómo arranca y se conecta el programa

Relación con la rúbrica: Main de arranque, distribución de responsabilidades y navegación.

Fragmento real: Proyecto2_PlantasVsZombies.java · líneas 5–8.

```
public static void main(String[] args) {

    javax.swing.SwingUtilities.invokeLater(() -> new Juego().iniciar());
}
```

• **main(String[] args):** es el punto por el que Java inicia la aplicación. args recibe posibles argumentos de ejecución.

• **invokeLater(...):** coloca la tarea de apertura en la cola del EDT, el hilo que atiende eventos y componentes de Swing.

• **() -> ...:** es una lambda: describe una tarea sin ejecutarla inmediatamente en esa misma expresión.

• **new Juego().iniciar():** construye el coordinador de inicio y solicita abrir la primera ventana.

Qué hace Juego

Su constructor crea GestorCuentas, que carga las cuentas existentes. iniciar() construye VentanaInicio con ese mismo gestor, centra la ventana y la hace visible. El main no contiene las reglas de combate.

Recorrido de llamadas que debes poder seguir

Acción	Clases que intervienen
Abrir la aplicación	main → Juego → GestorCuentas → VentanaInicio.
Crear o seleccionar cuenta	VentanaCrearCuenta / VentanaSeleccionarCuenta → GestorCuentas → Cuenta → VentanaMenuPrincipal.
Iniciar nivel	VentanaNuevaPartida → LectorNivel → ConfiguracionNivel → Jardin → Partida → MotorCombate → VentanaPartida.
Jugar	HilosPartida publica tareas → fotograma() → motor.actualizar(dt) → actualizar etiquetas y repaint().
Guardar y recuperar	VentanaPartida → GestorPartidas → archivo binario → reconstrucción de la ventana pausada.

> “El programa separa la presentación, los datos y las operaciones de archivos. La ventana recibe acciones; el modelo conserva el estado y aplica reglas; los gestores realizan la lectura y escritura.”

Pregunta de práctica

¿Por qué pasar GestorCuentas y Cuenta entre ventanas? Para mantener la cuenta activa y utilizar la información cargada, en lugar de perder el contexto al navegar. Crear una ventana no implica empezar otra partida.

2. Cuentas: validar, buscar y escribir

Relación con la rúbrica: Creación, unicidad sensible a mayúsculas, selección y archivos por cuenta.

Fragmento real: GestorCuentas.java · líneas 23–43.

```
public boolean nicknameValido(String nickname) {
    if (nickname == null || nickname.length() < 1 || nickname.length() > 12) {
        return false;
    }

    for (int i = 0; i < nickname.length(); i++) {
        if (!Character.isLetterOrDigit(nickname.charAt(i))) {
            return false;
        }
    }
    return true;
}

public Cuenta buscarCuenta(String nickname) {
    for (int i = 0; i < cantidadCuentas; i++) {
        if (cuentas[i].getNickname().equals(nickname)) {
            return cuentas[i];
        }
    }
    return null;
}
```

• **Primera condición:** rechaza null y longitudes fuera de 1–12. El cortocircuito de || evita consultar length() si la referencia es null.

• **for:** recorre cada carácter; isLetterOrDigit() admite letras o dígitos. Un carácter inválido devuelve false inmediatamente.

• **equals():** compara el contenido exacto del nickname. Ana1 y ana1 son diferentes.

• **Retornos:** buscarCuenta() devuelve el objeto encontrado o null. nicknameValido() devuelve true únicamente cuando pasa todas las comprobaciones.

Secuencia al pulsar Crear cuenta

El evento de VentanaCrearCuenta toma el texto y aplica trim(), que elimina espacios de los extremos. crearCuenta() valida, comprueba que no exista, prepara espacio en el arreglo y crea la carpeta, partidas y estadisticas.txt. Después agrega un objeto Cuenta y aumenta cantidadCuentas. Si falla la preparación de archivos devuelve false.

Rutas y recuperación

RutasCuentas: genera u- y cuatro dígitos hexadecimales por carácter. Ana se codifica u-0041006e0061; ana, u-0061006e0061. Esto evita depender de si el disco distingue mayúsculas. nickname() realiza la conversión inversa.

cargarCuentas(): enumera carpetas, recupera el nickname, carga las ocho líneas del archivo de estadísticas y crea Cuenta. El arreglo inicial puede ampliarse; solo cantidadCuentas posiciones contienen cuentas registradas.

> “La unicidad se comprueba sobre el texto exacto y la ruta conserva esa distinción. Cuenta relaciona el nickname con un objeto Estadisticas; GestorCuentas administra su persistencia.”

3. JSON: cómo funciona el analizador propio

Relación con la rúbrica: Lectura java.io, validación del formato y rechazo de niveles inválidos.

Entrada, proceso y salida

Entrada: un File elegido en JFileChooser. leerNivel() lee UTF-8 en bloques de caracteres, limita el tamaño y entrega el texto a Analizador. **Salida válida:** ConfiguracionNivel con duración, cerebros iniciales y String[][] del jardín. Una excepción se muestra como mensaje de nivel inválido.

Fragmento real: LectorNivel.java · líneas 19–30.

```
private static class Analizador {
    private final String texto; private int posicion;
    Analizador(String texto) { this.texto = texto; }
    private void espacios() {
        while (posicion < texto.length() && Character.isWhitespace(texto.charAt(posicion)))
            posicion++;
    }
    private boolean tomar(char c) {
        espacios();
        if (posicion < texto.length() && texto.charAt(posicion) == c) { posicion++; return true;
            }
        return false;
    }
    private void exigir(char c) { if (!tomar(c)) throw error("Se esperaba '" + c + "'"); }
```

• **texto y posicion:** conservan el contenido y el carácter que toca interpretar.

• **espacios():** avanza sobre espacios en blanco para que la presentación del JSON no afecte los separadores.

• **tomar(c):** consume el símbolo solo si coincide y devuelve si lo consiguió. Permite comprobar comas opcionales en los recorridos.

• **exigir(c):** requiere ese símbolo; si falta, genera una excepción con la posición del problema.

El resto del análisis

cadena(): exige comillas, recorre hasta el cierre y devuelve el texto. **entero():** recorre los dígitos y usa Integer.parseInt(). **clave():** comprueba el nombre esperado y los dos puntos.

leer(): exige las tres claves en el orden del formato; valida duración positiva y cerebros no negativos; recorre filas y plantas con do/while; compara el ancho de cada fila y exige el cierre completo sin contenido adicional.

Los arreglos temporales crecen cuando se llenan y al finalizar se copian a arreglos del tamaño exacto. valida() admite VACIO y los siete identificadores de plantas. No es un lector universal de cualquier variante de JSON: implementa el esquema del proyecto.

> “Si una fila tiene tres columnas y la siguiente cuatro, el analizador detecta la diferencia antes de crear Jardin. Así la construcción recibe una matriz rectangular válida.”

4. Jardín, casillas y estado del modelo

Relación con la rúbrica: Construcción dinámica, composición y organización de datos.

Fragmento real: Jardin.java · líneas 19–27.

```
public Jardin(String[][] configuracion) {

    filas = configuracion.length;
    columnas = configuracion[0].length;

    casillas = new Casilla[filas][columnas];

    crearJardin(configuracion);
}
```

Qué significa cada dato

• **configuracion.length:** cantidad de filas. configuracion[0].length es el ancho de la primera fila, ya validado por el lector.

• **new Casilla[filas][columnas]:** reserva la matriz de referencias; los objetos Casilla se crean dentro de crearJardin().

• **Dos for anidados:** el exterior cambia de fila y el interior recorre sus columnas. Para cada celda se crea una Casilla y se llama a crearPlanta().

• **VACIO:** crearPlanta() devuelve null. Los demás identificadores reconocidos producen el objeto de la planta concreta, que se coloca en la casilla.

Clase	Estado que representa
ConfiguracionNivel	Datos iniciales leídos del JSON. Sus dimensiones se obtienen de String[][].
Casilla	Fila, columna y referencia Planta. estaVacia() compara con null; quitarPlanta() retira esa referencia.
Jardin	Matriz Casilla[][], dimensiones, creación y siembra en espacios disponibles.
Partida	Cuenta, jardín, dificultad, reloj, cerebros, zombis, compras, gastos, podadoras y banderas de pausa/final.
MotorCombate	Coordina reglas y conserva proyectiles, cerebros visibles y plazos internos de los efectos.

Ejemplo de composición

> “La partida referencia un jardín; el jardín contiene casillas y una casilla puede contener una planta. Quitar la planta cambia el contenido de la casilla, no elimina la casilla ni cambia el tamaño del tablero.”

Pregunta: ¿por qué separar Partida y MotorCombate? Partida conserva el estado y operaciones de control; el motor coordina su evolución. La vista consulta ambos para mostrar los datos.

5. Del clic a la compra de un zombi

Relación con la rúbrica: Selección de tipo y carril, costo, validaciones y contadores.

1) Un botón del catálogo guarda el índice del tipo seleccionado. 2) PanelJardinFondo traduce la posición vertical del clic a fila. 3) La interfaz SeleccionCarril notifica a VentanaPartida.invocar(fila). 4) crear() construye el tipo concreto y se llama a Partida.invocarZombi().

Fragmento real: Partida.java · líneas 143–158.

```
public boolean invocarZombi(Zombi zombi) {
    if (zombi == null || pausada || terminada
            || zombi.getCosto() <= 0
            || zombi.getFila() < 0
            || zombi.getFila() >= jardin.getFilas()
            || costoActual(zombi.getCosto()) > cerebros) {
        return false;
    }

    guardarEnArreglo(zombi);
    int costo = costoActual(zombi.getCosto());
    cerebros -= costo;
    cerebrosGastados += costo;
    zombisComprados++;
    return true;
}
```

• **Condición inicial:** rechaza referencias nulas, pausa, partida terminada, costo no comprable, fila inexistente y falta de cerebros. Devuelve false sin modificar el estado.

• **guardarEnArreglo():** guarda la referencia del zombi. Si el arreglo está lleno, crea uno de doble capacidad y copia las posiciones existentes.

• **costoActual():** calcula el precio según el bloque de horda. Ese mismo valor se descuenta y se acumula como gasto.

• **zombisComprados++:** cuenta una compra exitosa. El true permite a la ventana actualizar la selección y el tablero.

Ejemplo con valores

Con 150 cerebros, sin descuento, comprar Caracono cuesta 100: el saldo queda en 50, cerebrosGastados sube 100 y zombisComprados sube 1. Intentar comprar después un Deportista de 150 devuelve false y mantiene esos valores.

> “El clic inicia la solicitud, pero la validación final está en Partida. La ventana informa el error si la compra no se puede realizar. agregarZombiGenerado() registra bailarines o Zombidito sin cobrar ni aumentar las compras.”

6. Reloj y actualización del motor

Relación con la rúbrica: Temporizador visible, movimiento continuo y coordinación de procesos.

VentanaPartida.fotograma()

System.nanoTime() mide el intervalo desde la última actualización. Se convierte de nanosegundos a segundos y se limita a 0.15. fraccion acumula pasos pequeños; por cada segundo completo se descuenta uno del reloj. El mismo dt avanza el motor.

> “Si fraccion vale 0.97 y llega un paso de 0.06, pasa a 1.03: se descuenta un segundo y quedan 0.03 para el siguiente ciclo. Por eso no se pierde el tiempo fraccionario entre fotogramas.”

Fragmento real: MotorCombate.java · líneas 96–113.

```
public void actualizar(double segundos) {
    if (segundos <= 0 || partida.estaPausada() || partida.estaTerminada()) return;
    segundos = Math.min(segundos, 0.15);
    ahora += segundos;
    actualizarEconomia();
    actualizarDefensa();
    for (int f = 0; f < animacionPodadoras.length; f++) {
        if (animacionPodadoras[f] > 0) {
            animacionPodadoras[f] = Math.max(0, animacionPodadoras[f] - segundos);
        }
    }
    asegurarEspacioZombis();
    activarPatatapumsYDigestion();
    dispararPlantas();
    moverProyectiles(segundos);
    moverYAtacarZombis(segundos);
    pinchohierbaYPatatapum();
}
```

• **Retorno inicial:** sin tiempo positivo o con pausa/final, no hay actualización.

• **ahora += segundos:** avanza el tiempo interno del motor. Los vencimientos se comparan con ese tiempo de juego.

• **Economía y defensa:** primero vencen cerebros y se evalúa si corresponde sembrar.

• **Estados, disparos y combate:** se preparan plantas, crean y mueven proyectiles, avanzan y atacan zombis y se resuelven reglas de casilla.

Después de actualizar, la ventana cambia etiquetas y solicita repaint(). Si el motor comunica victoria llama a finalizar(true); si el reloj llegó a cero llama a finalizar(false). El límite de dt evita un salto grande de simulación ante una demora, pero una demora prolongada no se recupera completamente como tiempo real.

7. Plantas: datos, herencia y plazos

Relación con la rúbrica: Las siete plantas, su daño, cadencia y estados gráficos.

Fragmento real: Lanzaguisantes.java · líneas 8–14.

```
public Lanzaguisantes(int fila, int columna) {

    super(100, fila, columna);

    danio = 20;
    cadenciaDisparo = 1000;
}
```

super(100, fila, columna): inicializa la parte heredada de Planta. danio y cadenciaDisparo pertenecen a Lanzaguisantes. La cadencia está en milisegundos; el motor la convierte a segundos al programar el siguiente disparo.

Clase	Cómo se ejecuta su comportamiento
Planta	Clase abstracta con salud y ubicación. recibirDanio() resta y limita la salud a cero; estaViva() comprueba si es positiva.
Lanzaguisantes	dispararPlantas() busca un blanco en dirección frontal, crea un Proyectil de daño 20 y programa el siguiente disparo a 1 segundo.
Hielaguisantes	Crea proyectiles de hielo con daño 15 cada 1.5 s. El impacto escribe congeladoHasta = ahora + 4; el avance se multiplica por 0.60 mientras ese plazo esté activo.
Patatapum	preparadaEn guarda cuándo se cumplen los 10 s. Entonces activar() cambia su bandera; al coincidir zombis en la casilla aplica 100 de daño y se retira.
Nuez	Tiene 400 de salud. obtenerEstadoVisual() devuelve 1, 2 o 3 por porcentaje; el panel representa grietas y daño. El motor impide que Garrocha la supere.
Carronivora	Si no está digiriendo, revisa su casilla y la frontal. Engulle o aplica 200 a Zombistein. digestionHasta marca ahora + 8; al vencer terminaDigestion().
Pinchohierba	No recibe mordiscos normales. proximoPincho programa 10 de daño cada 0.5 s en su casilla. Zombistein la retira mediante destruirPlanta().
Bipetidora	Busca blancos por separado a ambos lados. Crea un proyectil frontal y dos traseros de 15 de daño; utiliza una separación visual pequeña para la ráfaga.

> “Los estados temporales se guardan como plazos del reloj de juego. No se duerme durante ocho segundos dentro del ataque de Carroñívora; el motor sigue trabajando y comprueba cuándo termina su digestión.”

8. Zombis: estado particular y polimorfismo

Relación con la rúbrica: Catálogo, escudos, salto, baile, aplastamiento y lanzamiento.

La clase base Zombi

Conserva salud actual y máxima, daño, costo, modificador de velocidad, fila, posición y atacando. mover() solo cambia posicionX si está vivo y no ataca. perdioBrazo() compara salud con la mitad del máximo; el panel selecciona la imagen.

Clase concreta	Qué añade o cambia
ZombiComun	Inicializa el zombi base: 100 HP, daño 15, costo 50 y velocidad ×1.
ZombiCaracono	Escudo 100. Redefine recibirDanio() para consumir primero el escudo y getDanio() para usar 20 con cono o 15 sin él.
ZombiDeportista	Escudo 150 y velocidad ×2. Redefine el daño recibido; su ataque de 10 se programa cada 0.5 s.
ZombiGarrocha	tieneGarrocha y saltoRealizado impiden usos repetidos. realizarSalto() consume la garrocha y cambia la velocidad de ×1.7 a ×1.
ZombiBailon	Bandera de baile e intervalos. El motor detiene su avance durante 1 s e invoca adyacentes cada 5 s cuando hay espacio.
ZombiBailarin	Entidad secundaria creada por el Bailón; no pertenece al catálogo de compra.
Zombistein	Aplasta plantas. debeLanzarZombidito() exige salud a la mitad o menos y que todavía no haya lanzado. La bandera evita repetirlo.
Zombidito	Entidad secundaria de 50 HP y velocidad ×1.7; el motor conserva origen y plazo de su vuelo.

Ejemplo exacto del escudo

En recibirDanio(), si cantidad es menor o igual al escudo se resta solo del escudo. Si lo supera, se calcula danioRestante = cantidad − escudo, se pone el escudo en cero y se llama a super.recibirDanio(danioRestante).

> “Con 30 de escudo y un golpe de 50, quedan cero de escudo y se pierden 20 de salud. El motor utiliza una referencia Zombi, pero Java ejecuta el recibirDanio() del tipo real: ese es el polimorfismo.”

Detalle: ante Nuez el motor consume la garrocha y reduce la velocidad, pero no traslada al saltador al otro lado. Zombistein usa aplastamiento, no mordiscos; su daño base cero no significa que sea inofensivo.

9. Movimiento, colisiones y casilla lógica

Relación con la rúbrica: Movimiento continuo, primer impacto, ataques y reglas espaciales.

Mover y frenar delante de una planta

moverYAtacarZombis() recorre los zombis vivos. Busca la planta bloqueante más cercana por delante en el mismo carril; omite Pinchohierba. Calcula un avance base de 0.44 columnas/s × dt, aplica hielo y limita el movimiento para no atravesar el obstáculo. Zombi.mover() aplica además el modificador de velocidad de su clase.

> “Con dt = 0.10, el avance base es 0.044 columnas. Un Deportista lo multiplica por 2 y avanza 0.088 si no hay obstáculos. Con hielo se multiplica además por 0.60: avanza 0.0528.”

Impactos sin saltarse blancos

moverProyectiles() guarda la posición anterior, llama avanzar(dt) y obtiene el tramo recorrido. Revisa zombis vivos del mismo carril cuyo centro esté en el tramo. Conserva el de menor distancia al origen del paso, aplica daño y retira el proyectil. Los zombis en vuelo se excluyen de estos blancos.

Fragmento real: Proyectil.java · líneas 31–31.

```
public void avanzar(double segundos) { x += direccion * velocidad * segundos; }
```

direccion vale −1 o 1. velocidad vale 3 columnas/s. El modelo trabaja en columnas; el panel transforma esas medidas a píxeles. Esto conserva la lógica al cambiar el tamaño de la ventana.

Casilla única para efectos locales

Fragmento real: MotorCombate.java · líneas 473–476.

```
public int casillaActual(Zombi z) {
    int c = (int) Math.floor(z.getPosicionX() + 0.35);
    return Math.max(0, Math.min(jardin.getColumnas() - 1, c));
}
```

floor() obtiene el entero inferior de la posición del centro. Los límites evitan índices fuera de la matriz. Si posicionX es 1.8, el centro es 2.15 y la casilla es 2, usando índices desde cero.

Ataques y eliminación

atacarConMordisco() comprueba proximoMordisco: el común espera 1 s y Deportista 0.5 s. Una planta sin salud se retira y genera cerebros. Los zombis muertos permanecen como referencias en el arreglo de la partida, pero las reglas y el pintado los omiten; los proyectiles usados sí se retiran de su arreglo activo.

10. Cerebros: generación, vigencia y clic

Relación con la rúbrica: Generadores según dificultad, plazos aleatorios y recolección.

Cómo se representa un cerebro

Cerebro guarda x, y, vence y generador. x/y están en columnas y carriles; vence es un instante del reloj de juego. generador indica qué generador lo produjo; −1 identifica uno liberado por una planta destruida.

Fragmento real: MotorCombate.java · líneas 499–499.

```
private double esperaCerebro() { return 5 + azar.nextInt(11); }
```

nextInt(11) produce un entero entre 0 y 10; al sumar 5, el plazo queda entre 5 y 15 s, incluidos. El constructor crea proximoCerebro con 6, 5 o 4 posiciones según la dificultad e inicializa cada vencimiento por separado.

El ciclo de cada generador

• actualizarGenerador(indice) comprueba que la partida permita jugar y que haya vencido su plazo.

• Crea un Cerebro en una posición aleatoria y con vence = ahora + 5.

• Escribe Double.POSITIVE_INFINITY como próxima emisión mientras ese icono siga presente, evitando duplicarlo en cada actualización.

• Al recogerlo o vencer, quitarCerebro() programa una nueva espera aleatoria para ese generador.

Qué hace el clic

PanelJardinFondo revisa primero los iconos, del último al primero, para atender el que se dibuja encima. Si el clic cae en un cerebro, llama recogerCerebro(), solicita repaint() y retorna para no comprar también un zombi con el mismo clic.

recogerCerebro() rechaza pausa, final e índices inválidos. Si el icono ya venció, lo retira sin entregar recursos; si sigue vigente, lo retira y agrega 25 cerebros a Partida.

> “Con saldo 50, pulsar un cerebro vigente deja 75. Pulsarlo después de su vencimiento no agrega nada. La duración se mide en tiempo de juego, por eso la pausa también conserva su vigencia restante.”

Planta destruida

destruirPlanta() calcula 3 + nextInt(4): produce de 3 a 6 iconos. Cada uno vence a los 5 s y usa generador −1; retirarlo no reprograma ninguno de los generadores periódicos.

11. IA: cómo elige una única defensa

Relación con la rúbrica: Fácil, Medio y prioridades de Difícil.

Control del momento de actuar

actualizarDefensa() retorna si la dificultad es Fácil o todavía no llegó proximaDefensa. Si corresponde evaluar, programa el siguiente turno: 10 s en Medio o 5 s en Difícil. Esto evita sembrar en cada fotograma.

Medio

Elige uno de los siete tipos al azar. Recorre columnas desde la última hasta la primera y, dentro de cada columna, las filas. La primera siembra exitosa termina el método. Así busca desde el lado de la casa, que está a la derecha.

Difícil: regla exterior, zombis interiores

Un for recorre reglas 0–5 y otro revisa todos los zombis vivos. Esta estructura hace que se examine primero una prioridad en todo el tablero antes de considerar la siguiente. Si una acción consigue sembrar, return finaliza el turno.

Prioridad	Condición y acción del código
1. Cerca de la casa	Si quedan como máximo 4 columnas hasta la última, intenta Nuez o Carroñívora por delante.
2. Rebasó una planta	Si hay una planta detrás del zombi, intenta Bipetidora en un espacio detrás.
3. Resistente	Si es Caracono o Zombistein, intenta Patatapum o Carroñívora por delante.
4. Deportista	Intenta Hielaguisantes por delante.
5. Garrocha	Intenta Nuez por delante.
6. Más de 3 en carril	Cuenta zombis vivos del carril e intenta Pinchohierba.
7. Alternativa final	Cuenta defensas por fila y busca la menos defendida con espacio para Lanzaguisantes.

> “sembrarEnCarril() empieza en la casilla siguiente al zombi y busca espacios hacia la casa. sembrar() comprueba que la casilla esté libre e inicializa plazos de la planta nueva; por ejemplo, una Patatapum recién sembrada debe esperar sus propios diez segundos.”

Pregunta: ¿es una IA que aprende? No: utiliza selección aleatoria en Medio y reglas de prioridad en Difícil. Su comportamiento se puede explicar siguiendo condiciones y recorridos.

12. Bloques, descuentos, podadoras y final

Relación con la rúbrica: Siete periodos, hordas, podadoras de un uso y resumen.

Fragmento real: Partida.java · líneas 194–202.

```
public int getBloque() {
    double transcurrido = duracionTotal - tiempoRestante + fraccionReloj;
    return Math.min(7, 1 + (int)(transcurrido * 7.0 / duracionTotal));
}
// El enunciado no fija redondeo para costos fraccionarios: se redondea arriba.
public int costoActual(int base) {
    int bloque = getBloque();
    return bloque == 7 ? (base + 3) / 4 : bloque == 4 ? (base + 1) / 2 : base;
}
```

• **transcurrido:** duración inicial menos tiempo restante, más la fracción todavía no descontada.

• **× 7 / duración:** transforma el tiempo a una posición entre los siete bloques. Se suma 1 porque el primero se numera 1; min limita al bloque 7.

• **Precio:** el operador ternario elige cuarto de precio en bloque 7, mitad en bloque 4 o precio base. Con enteros, (base + 3) / 4 redondea hacia arriba.

Ejemplo con 70 s: de 0 a menos de 10 s es bloque 1; a los 30 s empieza el 4; a los 40 vuelve el precio normal en el 5; a los 60 empieza el 7. El común pasa de 50 a 25, vuelve a 50 y finalmente cuesta 13.

Llegada a la casa

llegadaALaCasa(fila) consulta el booleano de podadora del carril. Si está disponible, activarPodadora() lo cambia a false, suma el contador y el motor aplica daño suficiente a todos los zombis vivos de esa fila. La animación dura 0.9 s. Si ya se usó, victoria pasa a true y termina la partida.

Registro del resultado

VentanaPartida.finalizar() comprueba resultadoRegistrado antes de actuar. Luego marca el final, detiene hilos, actualiza estadísticas una vez, intenta escribirlas y muestra el resumen. La derrota se decide cuando el reloj alcanza cero sin invasión.

> “La primera llegada no gana: consume la defensa del carril. Para ganar hay que lograr otra llegada por un carril cuya podadora ya se haya usado.”

Demostración corta

En un JSON separado usa 70 s, 500 cerebros y una fila de cuatro VACIO. En Fácil, sin comprar, puedes mostrar los descuentos y la derrota. En otra partida, un común consume la podadora y otro posterior permite demostrar victoria. No cambies los niveles originales para esta prueba.

13. Hilos: trabajadores y EDT de Swing

Relación con la rúbrica: Concurrencia y restricciones técnicas; continuidad al pausar.

Fragmento real: HilosPartida.java · líneas 39–49.

```
private Thread trabajador(String nombre, int actual, Runnable tarea) {
    // La validacion se ejecuta en el EDT, no en el hilo trabajador.
    Runnable entrega = () -> { if (activo && sesion == actual) tarea.run(); };
    Thread hilo = new Thread(() -> {
        while (!Thread.currentThread().isInterrupted()) {
            try { Thread.sleep(40); }
            catch (InterruptedException e) { Thread.currentThread().interrupt(); return; }
            SwingUtilities.invokeLater(entrega);
        }
    }, nombre);
    hilo.setDaemon(true); hilo.start(); return hilo;
```

• **Runnable:** representa una tarea. entrega comprueba activo y el número de sesión antes de ejecutar tarea.run(). Esa comprobación ocurre en el EDT.

• **new Thread(...):** crea un trabajador real. El while continúa hasta una interrupción; sleep(40) evita que espere consumiendo CPU continuamente.

• **InterruptedException:** se restablece la marca de interrupción y se retorna para terminar el trabajador.

• **invokeLater(entrega):** publica el trabajo en la cola del EDT. El trabajador no modifica directamente el modelo ni los componentes.

• **daemon y start():** se marca como hilo auxiliar y se pone en ejecución; no se invoca run() directamente para arrancarlo.

Inicio y pausa

iniciar() exige EDT, evita un inicio duplicado, activa una sesión nueva y crea un reloj y 6/5/4 trabajadores generadores. Cada generador captura un índice final para actualizar su propio plazo.

detener() pone activo = false, incrementa sesion e interrumpe los hilos. Una tarea vieja ya encolada encuentra una sesión distinta y no actúa. Al reanudar se crean nuevos trabajadores usando los datos que permanecen en el modelo.

> “Los hilos son concurrentes, pero los cambios del modelo se atienden por turnos en el EDT. Los trabajadores despiertan cada 40 ms; eso no significa que produzcan cerebros cada 40 ms. La emisión depende del plazo aleatorio del motor.”

transient generadoresExternos: indica si la aplicación usa los trabajadores para revisar emisiones. No se serializa. El modo interno permite simulaciones y pruebas sin duplicar generadores cuando la interfaz está activa.

14. Pausa y guardado binario paso a paso

Relación con la rúbrica: Pausa completa, guardado por cuenta y recuperación pausada.

Pausar cambia tanto el modelo como los trabajadores

abrirPausa() pone pausada en true, detiene los hilos, deshabilita compras y abre el diálogo. Las operaciones del modelo también comprueban la pausa: no dependen solo del aspecto de los botones. Reanudar cambia la bandera, reinicia el instante de medición y crea trabajadores nuevos.

Fragmento real: GestorPartidas.java · líneas 25–35.

```
File destino = new File(carpeta, fecha + ".bin");
int sufijo = 1;
while (destino.exists()) destino = new File(carpeta, fecha + "-" + sufijo++ + ".bin");
File temporal = File.createTempFile("guardando-", ".tmp", carpeta);
try {
    try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(temporal))) {
        out.writeObject(new Estado(motor, fraccionSegundo));
    }
    if (!temporal.renameTo(destino)) throw new IOException("No se pudo completar el guardado.");
    return destino;
} finally { if (temporal.exists()) temporal.delete(); }
```

• **destino:** usa fecha/hora y extensión .bin. El while agrega un sufijo si ya existe otro con el mismo nombre.

• **temporal:** permite terminar la escritura antes de presentar el archivo como guardado completo.

• **ObjectOutputStream:** escribe un Estado que contiene motor y fraccionSegundo. Desde el motor se alcanzan la partida y sus entidades serializables.

• **try con recursos:** cierra el flujo incluso si ocurre una excepción. Tras escribir se renombra; finally limpia un temporal que haya quedado.

Qué se conserva y qué se reconstruye

Serializable permite guardar los campos de los objetos alcanzables: posiciones, salud, proyectiles, cerebros, podadoras, plazos internos y reloj. serialVersionUID identifica compatibilidad de serialización; Estado añade además una versión propia del formato.

cargar() usa ObjectInputStream y un filtro de clases y tamaño. Comprueba que haya un Estado válido, que el motor exista y la partida no esté terminada. vincularCuenta() exige el mismo nickname y la carga llama a pausar().

> “Se recuperan datos; no se recupera un Thread ejecutándose ni una ventana abierta. VentanaPartida se construye nuevamente con el motor recuperado y espera que el usuario pulse Reanudar.”

Prueba visible: guarda con un zombi o proyectil a mitad del tablero, vuelve al menú y carga. Compara tiempo, saldo y posiciones antes de continuar.

15. Estadísticas, archivo de texto y ranking

Relación con la rúbrica: Persistencia, porcentajes, promedio y tres Top 5.

Cuándo cambian los datos

Al finalizar, registrarVictoria(tiempo) o registrarDerrota() aumenta jugadas y el resultado correspondiente. Se agregan plantas destruidas, compras, gastos y podadoras de esa partida. Una victoria actualiza el mejor tiempo si es la primera o si fue más rápida.

Los resultados se guardan una sola vez mediante resultadoRegistrado. Guardar para continuar no se presenta como una victoria ni una derrota.

Cálculos

```
return (partidasGanadas * 100.0) / partidasJugadas;
return (double) plantasDestruidas / partidasJugadas;
```

Son las expresiones de getPorcentajeVictorias() y getPromedioPlantasDestruidas(); antes se comprueba si jugadas es cero. 100.0 y el cast double evitan la división entera que perdería decimales.

> “Con 3 victorias de 4 partidas, el porcentaje es 75%. Con 21 plantas destruidas en 4 partidas, el promedio es 5.25. Para rapidez se conserva el menor tiempo de una victoria, no el de una derrota.”

Cómo se persiste

escribirEstadisticas() escribe ocho líneas etiqueta: valor en estadisticas.txt. cargarEstadisticas() las lee en orden; leerValor() toma el texto después de los dos puntos y lo convierte a entero. El guardado comprobado escribe un temporal y reemplaza el destino.

Ordenamiento por inserción

Fragmento real: VentanaMenuPrincipal.java · líneas 77–81.

```
for (int i = 1; i < n; i++) {
    Cuenta c = ordenadas[i]; int j = i - 1;
    while (j >= 0 && comparar(c, ordenadas[j], tipo) < 0) { ordenadas[j + 1] = ordenadas[j]; j--;
        }
    ordenadas[j + 1] = c;
}
```

Se toma una cuenta desde la posición 1; mientras deba ir antes que la anterior, se desplaza esa anterior a la derecha. Después se coloca en el hueco. Por ejemplo, tiempos [40, 25, 35] quedan [25, 35, 40]. Se muestran como máximo cinco.

comparar() usa menor tiempo primero en rapidez, mayor cantidad primero en destrucción y victorias, y nickname para desempatar. El ranking de rapidez excluye cuentas sin victorias. El algoritmo opera sobre arreglos nativos.

16. Qué hacen las ventanas y el dibujo

Relación con la rúbrica: Swing, eventos, representación visual e interfaces.

Clase de Vista	Responsabilidad en el código
VentanaInicio	Presenta crear/seleccionar y abre la ventana correspondiente con el gestor.
VentanaCrearCuenta	Obtiene el texto, llama al gestor, muestra resultado y abre el menú con la cuenta creada.
VentanaSeleccionarCuenta	Llena el combo con las cuentas válidas; recupera la seleccionada y abre su menú.
VentanaMenuPrincipal	Navegación, lista de guardados, estadísticas y rankings.
VentanaNuevaPartida	JFileChooser, dificultad y creación de configuración, jardín, partida, motor y ventana.
VentanaPartida	Catálogo, etiquetas, acciones, reloj, pausa, guardado y resultado.
PanelJardinFondo	Transformación a píxeles, dibujo del tablero y entidades, selección de carril y recogida por clic.
HilosPartida	Ciclo de vida del reloj y trabajadores de generadores.
Recursos	Busca imágenes en rutas disponibles o dentro del JAR; las convierte y escala.
PanelFondo	Pinta un fondo escalado, lo centra y añade una capa oscura para legibilidad.

Cómo se convierte el clic en fila

```
int fila = (int) ((evento.getY() - area.y) / area.altoCasilla);
```

Se resta el borde superior del césped y se divide por la altura de una casilla. Con borde en 100, clic en 250 y casilla de 60 píxeles: (250 − 100) / 60 = 2.5; al convertir a int se obtiene la fila 2. Antes se comprueba que el clic esté dentro del césped.

Interfaz SeleccionCarril y representación

PanelJardinFondo informa la fila mediante alSeleccionarCarril(). La ventana registra una lambda que llama invocar(fila). El panel puede notificar sin conocer los detalles de compra.

repaint() solicita una actualización visual. Los métodos de dibujo leen posición y estado del modelo y seleccionan sprites, grietas, mensajes y barras. La imagen es una representación: cambiar un sprite no sustituye la lógica de salud o movimiento.

17. Consola y preguntas para defender el código

La consola no narra el combate

El juego es gráfico: saldo, tiempo, daño visual y resultado se muestran en Swing. La salida de NetBeans informa compilación, proceso Java y excepciones. BUILD SUCCESS o BUILD SUCCESSFUL corresponde al proceso de construcción o ejecución; no prueba por sí solo toda la funcionalidad.

GestorCuentas imprime algunos fallos de archivos. Los rechazos de compra o nivel se muestran mediante JOptionPane. Si se ejecutan las pruebas, aparecen sus mensajes OK; no se imprimen automáticamente al iniciar el juego.

Pregunta	Idea concreta para explicar
¿Qué significa this?	La referencia al objeto actual. this.cuenta = cuenta asigna el parámetro al atributo.
¿Qué significa super?	Invoca un constructor o una implementación de la clase padre.
¿Qué hace @Override?	Declara una redefinición de un método heredado; Java comprueba la relación.
¿Qué hacen return y continue?	return termina el método; continue pasa a la siguiente iteración del bucle. En la IA, return evita una segunda siembra.
¿Por qué null?	Representa ausencia de un objeto, como una casilla sin planta o una búsqueda sin cuenta.
¿Por qué instanceof?	Comprueba el tipo concreto antes de aplicar una regla especial o convertir la referencia.
¿Por qué final?	Impide reasignar esa variable. Si referencia un objeto, no implica que todo el objeto sea inmutable.
¿Por qué arreglos con contador?	La capacidad puede ser mayor que los elementos usados. El contador marca qué posiciones contienen datos activos.
¿Por qué serializar?	Para recuperar el grafo de datos y estados, no solo los valores que aparecen en las etiquetas.

Orden recomendado de demostración

Inicio → cuenta válida e inválida → selección → nivel JSON válido e inválido → compra y combate → recolección → pausa → guardar → recuperar → resultado → estadísticas. En cada paso muestra el método y explica entrada, condición y cambio de estado.

Documentación · 10 puntos

Muestra Manual_tecnico.pdf (UML, arquitectura, algoritmos, IA, hilos y archivos) y Manual_de_usuario.pdf (capturas del flujo). Esta guía es material de estudio para acompañar esos manuales. Practica siguiendo una acción completa entre clases, en lugar de memorizar únicamente los nombres de los métodos.